

PAPERBIT

Offline-First Peer-to-Peer Communication Platform

Technical Documentation

Document Overview

- 1.Executive Summary
- 2.Problem Definition
- 3.Goals and Design Principles
- 4.System Architecture
- 5.Communication Architecture
- 6.Data and Packet Model
- 7.Text Encoding and Compression
- 8.Audio Communication
- 9.Offline STT/TTS
- 10.Security and Privacy
- 11.Location and Context Sharing
- 12.Application Architecture
- 13.Reliability and Failure Handling
- 14.Performance Considerations
- 15.Testing and Evaluation Plan
- 16.Limitations and Future Work
- 17.References

1. Executive Summary

PaperBit is an offline-first communication platform intended for communication in connectivity-constrained environments. Instead of assuming continuous Internet access, the system treats nearby devices as communication participants that can exchange data directly and, where required, relay data across multiple devices. The design is centered on Bluetooth Low Energy (BLE), application-level forwarding, store-and-forward delivery, compact payload representation, optional low-bitrate audio, and on-device speech processing. The architecture is intended to degrade gracefully: when the network is sparse, messages can remain queued until another forwarding opportunity appears.

Primary design goals

- Operate without dependence on a centralized Internet service.
- Use nearby devices as communication opportunities and forwarding nodes.
- Minimize transmitted bytes and radio activity where practical.
- Support text as the lowest-cost communication mode.
- Support low-bitrate real-time voice as an optional higher-bandwidth mode.
- Provide offline speech-to-text (STT) and text-to-speech (TTS) using local inference.
- Protect message confidentiality and device/session identity through cryptographic mechanisms.
- Remain useful under intermittent connectivity and network partitioning.

2. Problem Definition

Conventional messaging applications are generally designed around stable IP connectivity and reachable backend infrastructure. This assumption breaks down during disasters, remote field operations, large infrastructure outages, crowded events, or intentionally disconnected environments.

Problem statement Design a mobile communication system capable of exchanging useful information between nearby participants when normal Internet connectivity cannot be assumed, while keeping bandwidth, energy, latency, and implementation complexity within practical limits.

Design constraints

- Radio range is limited and depends on the physical environment and device hardware.
- BLE payload exchange is substantially more constrained than conventional Internet transport.
- Nodes may appear and disappear frequently.
- A path to the destination may not exist at the time a message is created.
- Mobile devices have finite battery, memory, CPU, and thermal budgets.
- Voice traffic is substantially more expensive than compact text.

3. Goals and Design Principles

Offline-first The system should remain functional when the Internet is absent. Internet connectivity, if present, is an enhancement rather than a hard dependency.

Store-and-forward A node may store a message temporarily and forward it when a suitable peer becomes available. This converts intermittent contact into an asynchronous delivery mechanism.

Application-level mesh PaperBit can implement its own application-level forwarding protocol over BLE. This should not be described as implementing the complete Bluetooth Mesh specification unless the protocol actually conforms to that specification.

Bandwidth-aware design The system prioritizes compact text, packetization, deduplication, and optional compression before using more expensive media such as audio.

Local intelligence Speech recognition and synthesis are designed to run locally using an ONNX-compatible speech runtime such as sherpa-onnx, avoiding a mandatory cloud inference dependency.

4. System Architecture

At a conceptual level, PaperBit is divided into a user-facing application layer, a native communication layer, a storage/queue layer, and optional local AI/audio services.

Logical architecture User Interface → Application Service → Message/Media Encoder → Security Layer → BLE Transport → Peer Discovery / Session → Forwarding Queue → Nearby Peer

Major components Component

Responsibility

Status

Mobile UI

Conversation, delivery state, controls and local presentation.

Core

BLE discovery

Find nearby participating devices.

Core design

BLE data transport

Exchange application packets over nearby links.

Core design

Forwarding engine

Store, deduplicate and relay messages.

Core design

Local message store Persist outgoing, incoming and pending packets.

Core design

5-bit text encoder

Compact representation for the supported short-text alphabet. Current design

Brotli / LZ4

Compression candidates for payloads where overhead is worthwhile. Future/candidate

Audio codec

Opus or Codec2 for low-bitrate voice experiments.

Candidate

STT/TTS

Local speech recognition and synthesis.

Planned/feature

sherpa-onnx

Local speech inference runtime.

Selected technology

Location

Optional position/context sharing.

Feature

Cryptographic layer

Identity, key agreement and protected message sessions.

Design

5. Communication Architecture

5.1 Peer discovery

A participating device periodically advertises a recognizable application service identifier. Nearby devices scan for that service and determine whether a peer is eligible for communication. Discovery should be duty-cycled so that continuous scanning does not unnecessarily drain the battery.

5.2 Session establishment

After discovery, the peers establish a short-lived communication session. The session can negotiate protocol version, device capability, supported media types, maximum useful payload size, and security parameters.

5.3 Application-level forwarding

Messages are treated as application objects rather than as a continuous network stream. A node may receive, validate, persist, acknowledge, and later forward a message. This supports intermittent paths.

5.4 Store-and-forward lifecycle

1. Create a message and assign a globally unique message identifier.
2. Encode and protect the message.
3. Store the outgoing message locally.

4. Attempt transmission to eligible nearby peers.
5. Record acknowledgement or forwarding evidence where supported.
6. Retain the message while delivery remains possible and policy permits.
7. Discard it after expiry, successful delivery, or configured retry limits.

5.5 Duplicate suppression

Because a multi-hop environment may cause the same message to arrive through multiple peers, every message should carry a unique identifier. Nodes maintain a bounded recent-message cache. A packet whose identifier is already known can be acknowledged or ignored without reprocessing its payload.

6. Data and Packet Model

A compact binary envelope is preferable to repeatedly transmitting verbose JSON over a constrained radio link.

Illustrative packet structure VERSION | FLAGS | MESSAGE_ID | SOURCE_ID | DESTINATION_ID | TTL |
FLAGS

Delivery mode, acknowledgement, media and optional compression flags.

MESSAGE_ID

Unique identifier for duplicate suppression.

SOURCE_ID

Logical origin identifier.

DESTINATION_ID

Intended recipient or group identifier.

TTL

Limits uncontrolled propagation through the network.

SEQUENCE

Ordering/version information where required.

Field

Purpose

TYPE

Text, audio, location, control, acknowledgement, etc.

LENGTH

Payload length.

PAYLOAD

Encoded and, where applicable, compressed/encrypted content.

AUTHENTICATION TAG Integrity/authentication material.

TTL and bounded forwarding Every relayed packet should carry a bounded lifetime or hop budget. This prevents a message from circulating indefinitely in a dense network.

Message state Recommended local states are: CREATED → QUEUED → OFFERED → FORWARDED → ACKNOWLEDGED / EXPIRED / FAILED.

7. Text Encoding and Compression

7.1 Current compact text representation

PaperBit's current short-text strategy is a fixed-width 5-bit representation for a restricted supported character alphabet. With 5 bits, 32 distinct symbols can be represented. The encoder therefore needs an explicit character table and a defined fallback policy for unsupported characters. For example, if the application defines 32 symbols, each character maps to an integer in [0, 31], which is then packed bitwise into the payload.

Important implementation consideration: A fixed 5-bit alphabet is efficient only for the characters it directly represents. Unicode, punctuation, case changes, numbers, and arbitrary text require either additional modes, escaping, dictionaries, or a fallback encoding.

7.2 Compression strategy

General-purpose compression is not automatically beneficial for short messages. A compressor has metadata, window, framing, or entropy-model overhead. Therefore, PaperBit should select compression based on payload length and measured compressed size rather than compressing every message. •

Short text: use the compact 5-bit representation.

- Longer text: evaluate compression only when the compressed representation plus framing is smaller.
- Brotli: candidate for payloads where compression ratio matters more than minimal CPU cost.
- LZ4: candidate where very fast compression/decompression and low latency are more important than maximum compression ratio.
- Huffman coding: retained only as a possible future specialized optimization; it is not represented as a current PaperBit implementation.

8. Audio Communication

Audio is the most bandwidth-intensive communication mode and therefore should be treated differently from text. The intended pipeline is microphone capture → voice activity / framing → low-bitrate encoding → packetization → BLE transfer → reassembly → decoding → playback.

Codec candidates :

Codec

Role

Design consideration

Opus

Real-time voice/audio candidate.

Strong general-purpose interactive audio option; supports low latency

Codec2

Very-low-bitrate speech candidate. Useful where preserving intelligible speech matters more than naturalness

The final codec should be selected experimentally using bitrate, packet-loss tolerance, CPU cost, latency, and intelligibility as evaluation metrics. The documentation does not claim either codec is already integrated unless it is present in the implementation.

Packetization Encoded audio should be divided into independently identifiable chunks. Each chunk should carry enough metadata for ordering and reassembly. Loss of a single chunk should not necessarily invalidate the entire recording or call.

9. Offline STT/TTS

PaperBit can use local speech processing to convert speech to compact text before transmission. This can drastically reduce the amount of data required compared with sending raw or compressed audio, at the cost of local computation and model storage.

Speech-to-text path Microphone → PCM frames → local ASR/STT model → text → 5-bit encoding / optional compression → encrypted packet → BLE.

Text-to-speech path Received text → local TTS model → synthesized audio → local playback.

sherpa-onnx sherpa-onnx is the selected local speech-processing runtime direction. It provides ONNX Runtime-based speech processing and supports ASR and TTS models across platforms including Android. The exact pretrained model must be documented separately.

Pretrained model provenance For every shipped pretrained model, PaperBit documentation should record: exact model name, model author/organization, Hugging Face or upstream repository URL, original research paper where applicable, model license, and any additional dataset/model restrictions. Hosting a model on Hugging Face does not make Hugging Face the model's author. Model reference slot: [EXACT PRETRAINED MODEL NAME] — [AUTHOR/ORGANIZATION], [HUGGING FACE MODEL URL], [LICENSE], [ORIGINAL PAPER IF APPLICABLE].

10. Security and Privacy

Offline communication does not remove the need for security. In a peer-to-peer network, intermediate devices may observe, store, replay, duplicate, or attempt to modify packets unless the protocol protects against these actions.

Security objectives

- Confidentiality: intermediate forwarding nodes should not automatically learn message content.
- Integrity: a modified packet should fail authentication.
- Authenticity: the recipient should be able to determine whether a message came from an expected identity.
- Replay resistance: old valid packets should not be accepted as new events.
- Bounded metadata exposure: minimize unnecessary identity and location information.

Cryptographic architecture A modern design can use an authenticated key-establishment mechanism, derive symmetric session/message keys, and authenticate each packet with an AEAD construction. X25519 and HKDF

are examples of standardized primitives that can support key agreement and key derivation, but the exact cryptographic suite must be fixed before implementation. Security warning: Cryptography should not be custom-designed merely for the project. Use well-reviewed primitives, authenticated encryption, secure randomness, and a clear key lifecycle.

11. Location and Context Sharing

Location is an optional application payload rather than a requirement for the core messaging protocol. A location packet can contain latitude, longitude, timestamp, accuracy estimate, and an application-defined context. Location should be explicitly user-controlled because it can reveal sensitive information. The application should avoid silently forwarding precise location data beyond the intended audience. Illustrative location payload LAT | LON | ACCURACY | TIMESTAMP | CONTEXT_FLAGS

12. Application Architecture

The project direction uses a cross-platform application layer with native Android functionality exposed through native modules. This allows the UI and application state to remain manageable while radio, audio, and other platform-specific capabilities can be implemented close to the Android APIs.

Suggested module boundaries

Responsibility

UI / Presentation

Chats, contacts/peers, status, voice controls, settings.

Application State

Conversation state, peer state, delivery state.

Protocol

Encoding, packet parsing, flags, message IDs, TTL.

Security

Identity, session establishment, encryption/authentication.

BLE Transport

Scanning, advertising, connection, characteristic I/O.

Forwarding

Queue management, duplicate cache, retry policy.

Storage

Persistent local messages and metadata.

Audio

Capture, codec integration, playback.

Speech

sherpa-onnx model loading, STT/TTS inference.

Location

Location acquisition and payload creation.

No HTML/WebView renderer A WebView or local HTML renderer is intentionally excluded from the current PaperBit scope. It should not be presented as part of the implemented architecture unless the feature is added later.

13. Reliability and Failure Handling

- Peer disappears during transfer: retain unsent/unconfirmed data and retry later.
- Partial packet transfer: discard incomplete frame after timeout and retry at packet/chunk level.
- Duplicate packet: detect by message/chunk identifier and avoid duplicate application delivery.
- Stale packet: reject or expire using TTL/timestamp policy.
- Network partition: keep data locally until the retention policy is reached.
- Device restart: recover pending messages from persistent storage.
- Low battery: reduce scanning/advertising frequency and defer non-critical transfers.
- Unsupported feature: negotiate capabilities and fall back to a mutually supported representation.

14. Performance Considerations

Primary metrics Metric

Definition

Delivery latency

Time from message creation to successful recipient delivery.

Delivery ratio

Fraction of generated messages eventually delivered within the retention window.

Goodput

Useful application bytes delivered per unit time.

Overhead ratio

Protocol/framing bytes relative to useful payload bytes.

Energy per message

Battery energy attributable to discovery, connection and transfer.

Forwarding efficiency

Useful deliveries relative to relay transmissions.

Audio bitrate

Encoded voice bits per second.

STT latency

Time from speech input to usable recognized text.

TTS latency

Time from text availability to audible output.

Optimization priorities

- 1.Reduce unnecessary discovery and repeated advertising.
- 2.Avoid retransmitting already acknowledged or duplicate data.
- 3.Use compact binary framing.
- 4.Use 5-bit text representation for the supported short-text alphabet.
- 5.Use compression only when measured to reduce total transmitted bytes.
- 6.Prefer STT → compact text for situations where voice fidelity is not required.
- 7.Use low-bitrate speech/audio only when real-time or expressive audio is necessary.

15. Testing and Evaluation Plan

15.1 Functional tests

- One-hop text transmission between two devices.
- Multi-hop forwarding across three or more devices.
- Duplicate suppression.
- Message expiry and TTL handling.
- Device restart with queued messages.
- Peer disappearance and later reappearance.
- Unsupported-character handling in 5-bit text mode.
- Location payload creation and privacy controls.

15.2 Audio tests

- Codec encode/decode correctness.
- Packet loss and reordering behavior.
- Latency under different chunk sizes.
- Bitrate versus intelligibility.
- CPU and battery impact.

15.3 STT/TTS tests

- Recognition latency on target Android devices.
- Word error rate on representative speech samples.
- Behavior under background noise.
- Model memory footprint.
- TTS startup latency and sustained generation speed.
- Operation with airplane mode / no Internet access.

15.4 Network scenarios

Scenario

Expected observation

Two nearby devices

Low-latency direct exchange.

Three-node chain

Message can traverse an intermediate node.

Intermittent contact

Message remains queued and forwards when contact returns.

Dense peer environment

Duplicate suppression and bounded forwarding prevent uncontrolled replication.

No peers

Messages remain locally queued until expiry/policy action.

High packet loss

Retries and chunk-level recovery preserve useful delivery where possible.

16. Limitations and Future Work

Current limitations / design constraints

- BLE range and throughput vary significantly with device hardware and environment.
- A disconnected network cannot guarantee delivery when no temporal path to the destination exists.
- A 5-bit alphabet is inherently limited and requires a defined fallback for arbitrary Unicode text.
- Voice consumes substantially more bandwidth than text.
- On-device speech models consume storage, RAM, CPU and battery.
- The security design must be finalized and independently reviewed before claiming production-grade security.

Future enhancements

- Adaptive routing/forwarding based on encounter history.
- Priority-aware emergency messages.
- Smarter compression selection based on payload length and entropy.
- Huffman or other specialized coding for sufficiently long or structured text, if benchmarks justify it.
- Brotli/LZ4 integration for larger payloads.
- Opus and/or Codec2 integration with robust packet-loss concealment.
- More efficient model quantization for on-device STT/TTS.
- Optional infrastructure gateway mode when Internet connectivity becomes available.
- Formal protocol specification and interoperability test suite.

17. References

- [1] Bluetooth SIG, Bluetooth Core Specification, Bluetooth Special Interest Group.
- [2] Android Developers, “Bluetooth Low Energy,” Google. [Online]. Available: <https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview>
- [3] Bluetooth SIG, “Bluetooth Mesh Networking,” Bluetooth Special Interest Group. [Online]. Available: <https://www.bluetooth.com/learn-about-bluetooth/topology-options/le-mesh/>
- [4] V. Cerf et al., “Delay-Tolerant Networking Architecture,” RFC 4838, Internet Engineering Task Force, Apr. 2007.
- [5] A. Langley, M. Hamburg, and S. Turner, “Elliptic Curves for Security,” RFC 7748, Internet Engineering Task Force, Jan. 2016.
- [6] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, Internet Engineering Task Force, May 2010.
- [7] J. Alakuijala and Z. Szabadka, “Brotli Compressed Data Format,” RFC 7932, Internet Engineering Task Force, Jul. 2016.
- [8] Y. Collet, “LZ4 — Extremely Fast Compression,” LZ4 Project. [Online]. Available: <https://lz4.org/>
- [9] J.-M. Valin, K. Vos, and T. Terriberry, “Definition of the Opus Audio Codec,” RFC 6716, Internet Engineering Task Force, Sep. 2012.
- [10] J.-M. Valin et al., “Codec2,” Codec2 Project. [Online]. Available: <https://www.codec2.org/>
- [11] k2-fsa, “sherpa-onnx: Speech-to-Text, Text-to-Speech and Speech Processing with ONNX Runtime,” GitHub. [Online]. Available: <https://github.com/k2-fsa/sherpa-onnx>
- [12] Android Developers, “AudioRecord,” Google. [Online]. Available: <https://developer.android.com/reference/android/media/AudioRecord>
- [13] React Native, “Native Modules,” Meta Platforms, Inc. [Online]. Available: <https://reactnative.dev/docs/turbo-native-modules-introduction>

- [14] C. De Marco et al., “DisruptaBLE: Opportunistic BLE Networking,” in Proc. IEEE 47th Conf. Local Computer Networks (LCN), 2022, doi: 10.1109/LCN53696.2022.9843509.
- [15] C. Gomez et al., “How Low Energy Is Bluetooth Low Energy? Comparative Measurements with ZigBee/802.15.4,” in Proc. IEEE WCNC Workshops, 2012, doi: 10.1109/WCNCW.2012.6215496.
- [16] J.-M. Valin et al., “SoundStream: An End-to-End Neural Audio Codec,” IEEE/ACM Trans. Audio, Speech, Language Process., vol. 30, pp. 495–507, 2022, doi: 10.1109/TASLP.2021.3129994.
- [17] [MODEL AUTHOR/ORGANIZATION], “[EXACT PRETRAINED STT/TTS MODEL NAME],” Hugging Face. [Online]. Available: [MODEL URL].

Reference note: The pretrained model citation at [17] is intentionally left as a placeholder because the exact PaperBit model name and repository were not specified in the project discussion. It should be replaced with the actual model, its upstream source, applicable research paper, and license before final submission.